

# Theoretische examenvragen – met oplossingen

---

Gebaseerd op de “Snelle zelftest” uit de samenvatting, maar hier uitgewerkt met concrete antwoorden.

---

## Angular

1) Leg het verschil uit tussen interpolation, property binding en two-way binding.

### Antwoord

- **Interpolation** `{{ expr }}`: 1-richtings binding van **TypeScript** → **view** (tekst in de template). Je zet een expressie in de HTML.
- **Property binding** `[prop]="expr"`: 1-richtings binding **TypeScript** → **view**, maar dan naar een **property** van een DOM-element of component (bv. `[src]`, `[disabled]`, `[value]`).
- **Two-way binding** `[(ngModel)]="model"`: 2-richtings binding (view ↔ TS):
  - UI-wijziging past het model aan
  - modelwijziging update de UI
  - (typisch) vereist `FormsModule`.

Kort: interpolation is voor tekst, property binding is voor element/component properties, two-way is beide richtingen tegelijk.

2) Wanneer gebruik je `*ngIf` vs `ngSwitch`?

### Antwoord

- Gebruik `*ngIf` als je één conditie hebt: toon/verberg een stuk UI op basis van een boolean/conditie.
- Gebruik `ngSwitch` als je **meerdere exclusieve opties** hebt (zoals een switch-case): verschillende UI-blokken afhankelijk van een waarde (bv. `status = 'loading' | 'error' | 'ready'`).

Extra: `*ngIf` maakt het DOM-element echt aan/weg; `ngSwitch` is handig voor “exact één van vele” views.

3) Waarom zijn Observables handig voor HTTP calls?

### Antwoord

- `HttpClient` in Angular geeft Observables terug; dat past bij de rest van Angular/RxJS.
- Observables zijn **streams**: je kan er operators op chainen (`map`, `switchMap`, `catchError`, ...) en zo de dataflow declaratief maken.
- Je kan makkelijker omgaan met:
  - cancel/afbreken (praktisch via unsubscribe of `switchMap`-gedrag)
  - retries, timeouts, error handling
  - combineren van meerdere async bronnen.

4) Wat is het verschil tussen `providers` en `viewProviders` (met `<ng-content>`) ?

### Antwoord

- **providers**: de service is beschikbaar voor de component **én** voor **projected content** (content dat via `<ng-content>` "in" de component wordt geprojecteerd).
- **viewProviders**: de service is enkel beschikbaar in de **view** van de component (dus niet voor projected content).

Vuistregel: als projected content dezelfde dependency moet kunnen injecteren, gebruik **providers**.

## 5) Wanneer kies je template-driven forms vs reactive forms?

### Antwoord

- **Template-driven forms**:
  - sneller en eenvoudiger op te zetten
  - veel logica in de template
  - geschikt voor simpele forms
- **Reactive forms**:
  - model-driven: form state zit in TypeScript (**FormGroup**, **FormControl**)
  - beter voor complexe/dynamische forms
  - beter testbaar en voorspelbaar

Kort: klein & simpel → template-driven; complex/dynamisch/testbaar → reactive.

## 6) Leg het verschil uit tussen "subscribe in component" en "async pipe".

### Antwoord

- **Subscribe in component**:
  - je roept `.subscribe(...)` aan in TS
  - je moet meestal zelf opruimen (unsubscribe), zeker bij langlevende streams
  - geeft je volledige controle, maar sneller memory leaks als je cleanup vergeet
- **Async pipe** (`| async`) in de template:
  - Angular doet de subscribe/unsubscribe voor jou
  - code wordt vaak eenvoudiger (minder state-variabelen)
  - ideaal voor "toon observable data in UI"

Vuistregel: als je enkel wil renderen wat uit een Observable komt, is **async** meestal de beste default.

---

## NgRx / Redux

## 7) Beschrijf de flow: component → dispatch action → reducer → nieuwe state → selector → UI.

### Antwoord

1. **Component/UI** triggert een actie (user click, init, route change, ...)
2. Component **dispatcht** een **Action** naar de Store
3. **Reducer** ontvangt (**state**, **action**) en berekent een **nieuwe state** (immutably)
4. De **Store** bewaart die nieuwe state
5. **Selectors** selecteren/transformeren stukjes state en exposen die als Observables
6. UI **subscribet** (vaak met async pipe) en rendert de nieuwe data

## 8) Waarom moeten reducers “pure” zijn?

**Antwoord** Een pure function:

- geeft voor dezelfde input altijd dezelfde output
- heeft geen side effects

Waarom belangrijk:

- voorspelbaarheid en makkelijker debuggen
- beter testbaar (unit tests)
- geen onverwachte mutaties of async gedrag in reducers

Side effects (HTTP, timers, random, storage) horen in **effects** of services, niet in reducers.

## 9) Geef 3 voorbeelden van state die wél in de Store hoort (SHARI) en 3 die niet hoort.

**Antwoord Wel in Store (SHARI-achtig):**

- Ingelogde gebruiker + rollen/permissions (Shared, Available, Impacted)
- Productenlijst die door meerdere screens wordt gebruikt (Shared, Retrieved)
- UI voorkeuren die je wil bewaren (theme, taal) (Hydrated)

**Niet in Store:**

- Tijdelijke input in een enkel formulier (Angular form state)
- “Is dropdown open?” in één component (lokale UI micro-state)
- Niet-serialiseerbare objecten (bv. class instances met methods, DOM refs)

## 10) Welke acties maak je typisch rond async: **load**, **loadSuccess**, **loadFailure**?

**Antwoord** Een standaard async patroon:

- **loadX**: start het ophalen (effect luistert hierop)
- **loadXSuccess**: effect dispatcht dit met de opgehaalde data
- **loadXFailure**: effect dispatcht dit met error info

Waarom:

- UI kan loading/error states tonen
- reducer blijft pure (alleen state transitions)

---

## React

## 11) Wat is Virtual DOM en wat is reconciliatie?

**Antwoord**

- **Virtual DOM**: een in-memory representatie van de UI (een “boom” van elementen).
- **Reconciliatie**: React vergelijkt de vorige virtual DOM met de nieuwe (na state/props change) en bepaalt de minimale set wijzigingen om de echte DOM te updaten.

Doel: efficiënt updaten en een declaratieve programmeerstijl.

12) Waarom is `useEffect` nodig voor side effects (timer/fetch) en wat doet cleanup?

#### Antwoord

- Render-functies moeten idealiter “pure” blijven: UI beschrijven op basis van state/props.
- Side effects zoals:
  - timers (`setInterval`)
  - subscriptions
  - data fetches horen in `useEffect`.

**Cleanup** (return-functie in `useEffect`) ruimt op:

- interval clearen
- subscription unsubscribe
- event listeners verwijderen

Zonder cleanup krijg je vaak memory leaks of dubbele timers/subscriptions.

13) Wat is het verschil tussen props en state?

#### Antwoord

- **Props:** input voor een component, wordt van buitenaf meegegeven (parent → child). In principe read-only binnen de child.
- **State:** interne data van de component die kan veranderen (via `setState/useState`). State changes triggeren re-render.

14) Geef een voorbeeld van een dependency array bug in `useEffect`.

**Antwoord** Klassieke bug: je gebruikt een variabele in `useEffect`, maar zet die niet in de dependency array.

Voorbeeld (conceptueel):

- Je effect gebruikt `userId` om te fetchen.
- Je zet `[]` als dependencies.

Gevolg:

- Het effect draait maar 1x (bij mount).
- Als `userId` verandert, blijft de component oude data tonen.

Oplossing:

- `userId` opnemen in de dependency array, of de logica herschrijven (bv. via memoization / callbacks) zodat dependencies kloppen.

---

## ASP.NET Web API

15) Waarom DTO's gebruiken (over-posting)?

**Antwoord**

- Zonder DTO kan een client extra velden meesturen die je niet wil laten aanpassen (bv. `isAdmin`, `secret`, `price`, ...).
- Dat heet **over-posting**: ongewenste properties worden toch gebonden naar je model.

DTO's lossen dit op:

- Je definieert exact welke velden je accepteert voor create/update.
- Je mappt DTO → entity (en omgekeerd voor responses).

**16) Wat doet Swagger/OpenAPI voor je API?****Antwoord**

- Swagger/OpenAPI genereert **machine-leesbare API documentatie** (endpoints, models, responses).
- Swagger UI geeft een **interactieve pagina** waar je endpoints kan uitproberen.

Voordelen:

- sneller testen/debuggen
- duidelijke contracten voor frontend/back-end samenwerking

**17) Hoe test je endpoints met HttpRepl?****Antwoord**

- Installeer: `dotnet tool install -g Microsoft.dotnet-httprepl`
- Start: `httprepl <baseUrl>`
- Test: `get /api/...`, `post /api/...` enz.

Doel: snel API calls testen vanuit een interactieve REPL.

**18) In MongoDB: wat is het verschil met relationele DB's (documenten/collections)?****Antwoord**

- Relatieel (SQL): tabellen + rijen + relaties/joins, schema is meestal strak.
- MongoDB (NoSQL document):
  - **collections** i.p.v. tabellen
  - **documents** (JSON/BSON) i.p.v. rijen
  - schema is flexibeler (maar je moet nog steeds afspraken maken)
  - relaties worden vaker via embedding of referenties opgelost i.p.v. klassieke joins

---

## Build pipeline

**19) Leg uit wat transpilation vs bundling is.****Antwoord**

- **Transpilation**: syntax omzetten (TS/JSX/ESNext → JS) zodat tooling/browsers het begrijpen.

- **Bundling:** imports volgen en alles samenvakken in 1 of meerdere output bundles/chunks.

20) Wanneer gebruik je code splitting/lazy loading?

**Antwoord**

- Als je initiële bundle te groot wordt en je sneller “first paint” wil.
- Als je delen van de app pas later nodig hebt:
  - routes (admin pagina’s)
  - zware componenten (charts)
  - zelden gebruikte flows

21) Wat is tree-shaking en wanneer kan het mislopen?

**Antwoord**

- Tree-shaking verwijdert ongebruikte exports uit je bundles.
- Het kan mislopen bij:
  - modules met **side effects** (code die bij import al iets doet)
  - niet-ESM patterns of dynamische requires/imports
  - “barrel” exports waardoor tooling minder precies kan zijn

Vuistregel: schrijf zo veel mogelijk met ESM imports/exports en markeer side-effect free modules correct in tooling.